

3.Process Management

3.1 Process Definition, Process states

3.2 Process Control Block

3.3 Scheduling Criteria

3.4 Process Scheduling: Definition, Scheduling objectives

3.5 Scheduling algorithms(FCFS, SJF, Priority Scheduling, Round-Robin Scheduling)

Process :-

A process is basically a program in execution. The execution of a process must progress in a sequential fashion.

A process is defined as an entity which represents the basic unit of work to be implemented in the system.

When we save a program it saves in secondary memory and it is called a program and when we run it it comes in main memory for execution then it is called as process.

Example:-

Saved C file → Program

Running C file → Process

Program vs Process :

Program :-

- Stored on secondary memory (hard disk / SSD)
- Passive (not running)
- Example: hello.c
- Program → Static entity
- **Static entities are just stored instructions**
- Example: A program saved on your computer.

Process :-

- Loaded into main memory (RAM)
- Actively executing
- Has stack, heap, registers
- Process → Dynamic entity
- **Dynamic entities are running instances**
- Example: A process created when you run the program.

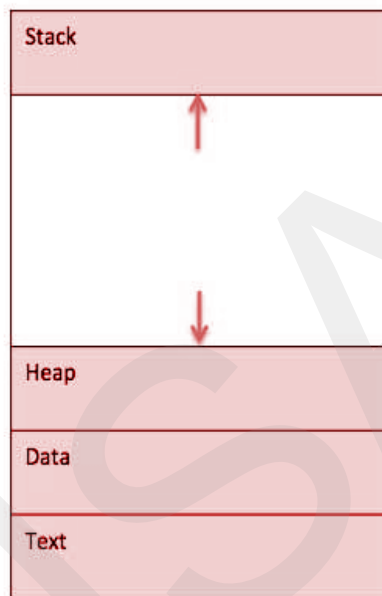
Real-Life Example :-

Static: A recipe written in a book

Dynamic: Cooking the recipe in the kitchen

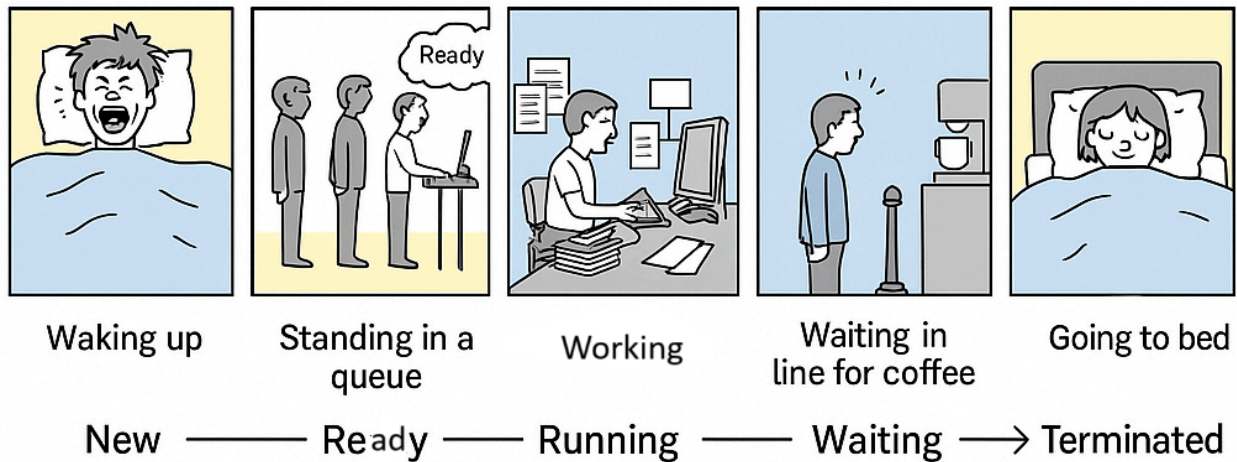
When a program is loaded into the memory and it becomes a process, it can be divided into four sections : stack, heap, text and data.

The following image shows a simplified layout of a process inside main memory –



1. **Stack** :- The process Stack contains the temporary data such as method/function parameters, return address and local variables.
2. **Heap** :- This is dynamically allocated memory to a process during its run time.
3. **Text** :- This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.
4. **Data** :- This section contains the global and static variables.

Process states :-



When a process executes, it passes through different states. These stages may differ in different operating systems.

1. Start :-

This is the initial state when a process is first started/created .It is present in secondary memory.

The process is being created. The OS is setting up the necessary resources and the Process Control Block (PCB) but has not yet loaded it into main memory.

The process is not in main memory.

2. Ready :-

The process is loaded into main memory and has everything it needs. It is in a queue, waiting to be assigned to a processor to run. There is a queue called the READY QUEUE in which all these processes are waiting for the processor . Many processes can be in the ready state at once.

3. Running :-

Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.

The process's instructions are being executed by the CPU. On a single-core CPU, only one process can be in the running state at any given moment.

4. Waiting :-

Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.

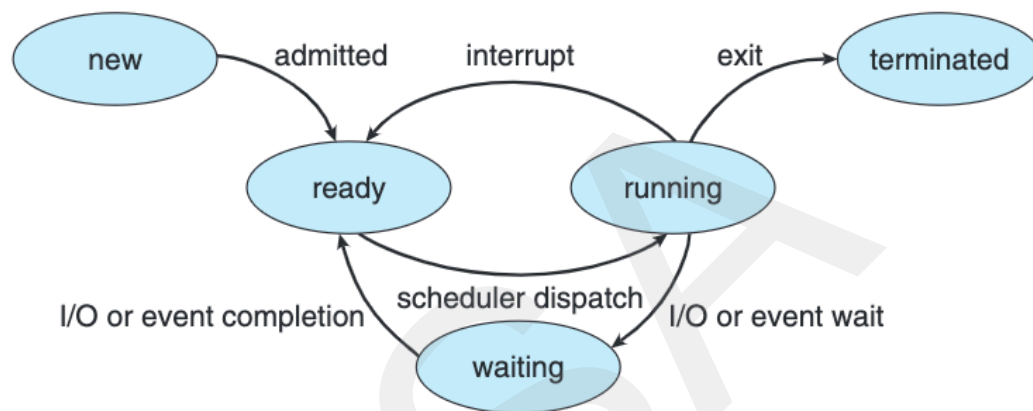
Even if the CPU is free, a waiting process cannot run until its event is complete.

5. Terminated or Exit :-

Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state . The OS will now deallocate all the resources that were assigned to it.

How Do Processes Move Between States?

Here's a simple version of the state transition diagram:



- A process begins in the New state when it is created.
- It moves to Ready when it's prepared to run but is waiting for CPU time.
- Once scheduled, it enters the Running state where the CPU executes it.
- If the process requires I/O, it transitions to Waiting until I/O is completed.
- After I/O completion, it returns to Ready and waits for CPU time again.
- When the process finishes its execution, it moves to the Terminated state and is removed from memory.

Process Control Block :-

A Process Control Block (PCB) is a data structure maintained by the operating system for every process.

It contains all the information about a process.

PCBs are stored in specially reserved memory of Operating Systems known as kernel space.

PCB refers to data structure that keeps track of information about a specific process.

The CPU requires this information to complete the job.

Each process has its own PCB that identifies it.

Important Point:-

- Each process's PCB is stored in main memory.
- Each process has only one PCB associated with it.
- All of the processes PCBs are listed in the linked list.

Role of Process Control Block:-

It's a job of the Operating System to assign CPU to a process as the process doesn't need CPU all the time.

For example, input/output processes are only used by the CPU when they are triggered.

The role of Process Control Block arises as an identification card for each process.

The operating system doesn't know which process is which until the operating system refers through the PCB for every process.

For example,

There are MS Word processes, printing processes, and many background processes running currently on the CPU.

How will the OS identify and manage each process without knowing the identity of each process.

So here PCB comes into play as a data structure to store information about each process.

Therefore, whenever a user triggers a process (like print command), a PCB created for that process in the Operating System which is used by the operating system to execute and manage the processes when the operating system is free.

Structure of Process Control Block:-

The process control block contains many attributes such as process ID, process State, process priority, accounting information, program counter, CPU registers etc for each process.

Process ID
Process state
Process priority
Accounting information
Program counters
CPU registers
PCB pointers
List of open files
Process I/O status Information
.....

SCALER
Topics

1. Process ID

When a new process is created by the user, the OS assigns unique ID i.e. Process ID to that process.

This process ID helps processes to be distinguished from other processes that exist in the system.

2. Process State

A process passes through different states from creation to completion, generally including these 5 states :

- New: Processes ready to be loaded into main memory.
- Ready: Processes in the main memory (RAM) waiting to be assigned to the CPU for execution.

- **Running:** The state where instructions are executed.
- **Waiting:** Processes waiting for I/O or event completion.
- **Terminated/Exit:** Process completion.

Transitions Mentioned:

- **Admitted:** Moves a process from New to Ready .
- **Scheduler Dispatch:** Moves a process from Ready to Running.
- **I/O or event wait:** Moves a process from Running to Waiting .
- **I/O or event completion:** Moves a process from Waiting to Ready.
- **Interrupt:** Moves a process from Running to Ready .
- **Exit:** Terminates the process .

3. Process Priority:

A numeric value that represents the priority of each process. This priority is assigned at the time of PCB creation (or by a user) and may depend on many factors like age of that process, resources consumed and so on.

It determines the importance of the process ; generally lower value indicates higher priority.

4. Process Accounting Information:

It gives the information about the resources used by the process in its lifetime, such as CPU time and connection time, over the process's lifetime .

5. Program Counter:

A pointer indicating the address of the next instruction to be executed for the process. This attribute contains the address of next instruction to be executed in the process.

6. CPU Registers:

Small, fast storage locations in RAM where register data is stored.
CPU registers are a quickly accessible small sized location available to the CPU.

7. PCB Pointer:

Contains the address of the next PCB in the ready state, allowing the OS to manage control flow between parent process and child process.

8. List of Open Files:

Keeps track of files used by the process to ensure they are closed at the time of termination of the process.

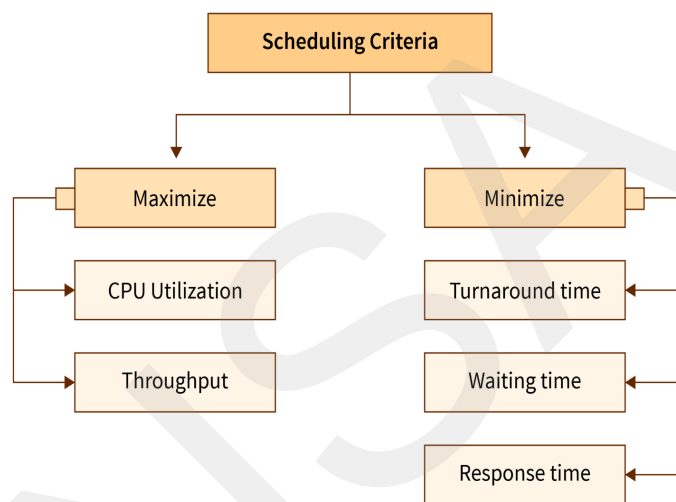
9. Process I/O Information :-

In this field, the list of all the Input/Output Devices which are required by that process during its execution is mentioned.

PCBS are stored in the form of Linked List in memory as :-

PCB1 -> PCB2 -> PCB3

Process Scheduling criteria and objectives:-



SCALER
Topics

Process scheduling criteria in operating systems are metrics used to evaluate and compare scheduling algorithms to ensure efficient, fair, and fast system performance.

The primary goals are to maximize CPU utilization and throughput, while minimizing turnaround time, waiting time, and response time

The primary goal of these criteria is to ensure the system is efficient, fast, and fair.

Process Scheduling criteria :-

Criteria = Measurement parameters

These are the factors used to evaluate how good a scheduling algorithm is.

1. CPU Utilization (Maximize) :-

Keep the CPU as busy as possible.

CPU utilization is a criterion used in CPU scheduling that measures the percentage of time the CPU is busy executing a task.

It is important to maximize CPU utilization because when the CPU is idle, it is not performing any useful work, and this can lead to wasted system resources and reduced productivity.

A high CPU utilization indicates that the CPU is busy and working efficiently, processing as many tasks as possible.

2. Throughput (Maximize):-

Number of processes completed per unit time.

Throughput measures the number of tasks or processes completed within a specific period.

It is important to maximize throughput because it reflects the efficiency and productivity of the system.

3. Turnaround Time (Minimize):

Total time from submission to completion.

Turnaround time is the time it takes for a process to complete from the time it is submitted to the system until it is fully processed and ready for output.

It is important to minimize turnaround time because it reflects the overall efficiency of the system and can affect user satisfaction and productivity.

Formula:

Turnaround Time = Completion Time - Arrival Time

4. Waiting Time (Minimize) :-

Time a process waits in the ready queue.

Waiting time is a criterion used in CPU scheduling that counts the amount of time a task or process waits in the ready queue before it is processed by the CPU.

Formula:

Waiting Time = Turnaround Time - Burst Time

5. Response Time (Minimize) :-

Time from request submission to first response.

Response time is a criterion used in CPU scheduling that measures the time it takes for the system to respond to a user's request or input.

It is important to minimize response time because it affects user satisfaction and the overall efficiency of the system.

Process Scheduling Objectives :-

Objectives = Goals to achieve These criteria

These are the aims of the scheduling system.

They tell us what we want to improve.

Common Objectives :-

1. Maximize CPU utilization
2. Maximize throughput
3. Minimize waiting time
4. Minimize turnaround time
5. Minimize response time

Scheduling algorithms :-

1) First Come First Serve (FCFS) Scheduling :-

- FCFS is a non-preemptive scheduling algorithm in which:
- The process that arrives first is executed first.
- This is a simple and easy to implement algorithm.
- When to choose this algorithm
- It is easy to implement and requires minimal overhead for tracking processes.

Advantages :-

1. It is simple to implement, making it easier for OS designers and students to understand and implement.
2. Processes are served in the exact order they arrive.

Disadvantages :-

1. If a large process that has a long burst time arrives first, all the smaller processes that come after will have to wait for its completion.
2. This increases the average waiting time and turnaround time, making the CPU inefficient.

Example :-

Given Data:-

Process	Burst Time (ms)
P1	5
P2	3
P3	8

(Assume Arrival Time = 0 for all processes)

Execution Order

P1 → P2 → P3

Gantt Chart :

P1	P2	P3	
0	5	8	16

Calculations :-

- **Turnaround Time (TAT) :-**
- $TAT = \text{Completion Time} - \text{Arrival Time}$
- **Waiting Time (WT)**
- $WT = \text{Turnaround Time} - \text{Burst Time}$

FCFS Calculation Table :-

Process	Burst Time	Completion Time	TAT	WT
P1	5	5	5	0
P2	3	8	8	5
P3	8	16	16	8

Average Times :-

Average WT :- $(0+5+8)/3=4.33$

Average TAT :- $(5+8+16)/3=9.66$

FCFS Result :-

FCFS is simple but results in higher waiting time, especially for longer processes.

2) Round Robin (RR) Scheduling :-

Round Robin is a preemptive scheduling algorithm in which:

Characteristics :-

1. Fair scheduling
 2. Used in time-sharing systems
 3. Performance depends on time quantum
- In the round-robin scheduling algorithm, each process gets a fix amount of CPU time called a time slice or time quantum for execution.
 - The CPU gives a fixed time to each process.
 - The process executes for its time slice and is preempted and then the next process in the queue is executed for a given time period.
 - If the process completes its burst time, the process releases the CPU; otherwise, the process is added at the end of the ready queue and gets a chance to execute again.
 - Context switching is used to save states of preempted processes.

Characteristics :-

1. Fair scheduling :- The round-robin algorithm is chosen to provide fair CPU allocation to each process.
2. Used in time-sharing systems
3. Performance depends on time quantum

4. It prevents starvation and a single process from having the CPU for too long, making it suitable for multitasking environments.

Advantages :-

1. Every process gets a chance to run, preventing starvation and ensuring that all processes receive some CPU time.
2. The system becomes more responsive because Round Robin gives each process time, as no process waits indefinitely.

Disadvantages :-

The performance of the CPU is highly dependent on the time slice. If the time slice is too long, Round Robin starts behaving like FCFS, and if it is too small, the excessive context switching increases overhead.

Given Data (Example) :

Process	Burst Time (ms)
P1	5
P2	3
P3	8

(Assume Arrival Time = 0 for all processes)

Time Quantum (TQ) = 2 ms

Step-by-Step Execution :-

Cycle	Process	Remaining Burst
1	P1	5 → 3
2	P2	3 → 1
3	P3	8 → 6
4	P1	3 → 1
5	P2	1 → 0 (Done)
6	P3	6 → 4

7	P1	1 → 0 (Done)
8	P3	4 → 2
9	P3	2 → 0 (Done)

Gantt Chart (Round Robin) :-

P1	P2	P3	P1	P2	P3	P1	P3	P3	
0	2	4	6	8	10	12	14	16	18

Completion Times(Finish Time) :-

Process	Completion Time
P1	14
P2	10
P3	16

RR Calculation Table :-

Process	Burst Time	Completion Time	TAT	WT
P1	5	14	14	9
P2	3	10	10	7
P3	8	16	16	8

Average Times :-

Average WT : $(9+7+8)/3=8$

Average TAT : $(14+10+16)/3=13.33$

Imp.points :-

1.Non-Preemptive :-Once a process starts, it runs until completion.(Cannot be interrupted)

2.Preemptive :- a process starts,can be stopped if other process with high priority or shortest remaining time arrives(Can be interrupted)

3.priority :-

Means he has high priority and other common people have low priority.

High priority gets executed first and low priority gets executed afterwards.

Think of Priority Scheduling like a VIP line at a club or an Emergency Room at a hospital. It doesn't matter who got there first; the most "important" person gets served first.

How it Works :-

In an operating system, every task (process) is given a priority number.

- **High Priority:** These tasks get the CPU immediately.
- **Low Priority:** These tasks have to wait until the "important" ones are finished

Terminologies Used in CPU Scheduling

- 1. Arrival Time (AT):** This is the time when a process initially enters the ready queue.
- 2. Burst Time (BT):** The burst time is also known as Execution or CPU time. It is the time in seconds or milliseconds that a process requires to complete its execution. This is often an estimated value used by scheduling algorithms.

3. **Completion Time (CT):** This is the exact time when the process completes and leaves the CPU.
4. **Turnaround Time (TAT):** The total time a process stays within the system, from the arrival time to its completion.

$$\text{TAT} = \text{CT} - \text{AT} \text{ (Turnaround Time = Completion Time - Arrival Time)}$$

5. **Waiting Time (WT):** It is the time a process spends waiting in the ready queue for its turn to access the CPU. This excludes time spent doing I/O.

$$\text{WT} = \text{TAT} - \text{BT} \text{ (Waiting Time = Turnaround Time - Burst Time)}$$

6. **CPU Utilization:** The percentage of time that the CPU is actively busy executing processes versus the time it is sitting idle doing nothing.
7. **Time Slice:** This term is used in preemptive scheduling, which is the time a process is allowed to run on the CPU before being preempted.

2. Shortest Job First:-

- The shortest job first scheduling algorithm works on the principle of giving the CPU to the process that has the shortest burst time.
- The process with the smallest burst time(finish time) is executed first.
- It minimizes average waiting time compared to FCFS.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processor should know in advance how much time the process will take.

It can be both preemptive and non-preemptive.

Types of SJF :-

1. Non-Preemptive SJF :-

Once a process starts, it runs until completion.(Cannot be interrupted).

Non-preemptive SJF reduces waiting time but requires knowing burst time in advance.

2. Preemptive SJF (SRTF – Shortest Remaining Time First) :-

A running process can be stopped if a new process with shorter remaining time arrives.(Can be interrupted).

When it is preemptive, it becomes the Shortest Remaining Time First (SRTF) Scheduling Algorithm.

Characteristic :-

Operating systems choose SJF when they need minimum average waiting time and minimum average turnaround time for a given set of processes.

Advantages of SJF

- Minimum average waiting time
- Efficient CPU utilization

Disadvantages of SJF

- Burst time must be known in advance
- Starvation of long processes

The processes with longer wait times might be put on hold for a long time before they get executed, which might lead to starvation of these processes.

- Difficult to implement practically

Important Formulae

- **Turnaround Time (TAT)**

$TAT = \text{Completion Time} - \text{Arrival Time}$

- **Waiting Time (WT)**

$WT = \text{Turnaround Time} - \text{Burst Time}$

PART A: Non-Preemptive SJF Scheduling

Given Data :-

Process	Arrival Time	Burst Time
P1	0	0
P2	0	2
P3	0	8
P4	0	3

(All processes arrive at time 0)

Execution Order (Shortest Job First) :-

$P2 \rightarrow P4 \rightarrow P1 \rightarrow P3$

Gantt Chart :-

P2	P4	P1	P3	
0	2	5	11	19

Completion Time, WT and TAT Calculation :-

Process	BT	CT	TAT	WT
P2	2	2	2	0
P4	3	5	5	2
P1	6	6	11	5
P3	8	8	19	11

Average Waiting Time : $(0+2+5+11)/4=4.5$

Average Turnaround Time : $(2+5+11+19)/4=9.25$

Part B : Preemptive SJF (SRTF – Shortest Remaining Time First) :-

A running process can be stopped if a new process with shorter remaining time arrives.(Can be interrupted)

Given Data :-

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Step-by-Step Execution :-

1. Time 0–2: P1 runs
2. Time 2: P2 arrives (shorter than P1 → preempt)
3. Time 2–4: P2 runs
4. Time 4: P3 arrives (shortest → preempt)
5. Time 4–5: P3 completes

6. Time 5–7: P2 completes
7. Time 7–11: P4 runs
8. Time 11–16: P1 completes

Gantt Chart (Preemptive SJF) :

P1	P2	P3	P2	P4	P1	
0	2	4	5	7	11	16

Completion Time, WT and TAT :

Process	AT	BT	CT	TAT	WT
P1	0	7	16	16	9
P2	2	4	7	5	1
P3	4	1	5	1	0
P4	5	4	11	6	2

Average Waiting Time : $(9+1+0+2)/4=3$

Average Turnaround Time : $(16+5+1+6)/4=7$

Priority Scheduling :-

Priority scheduling is a CPU scheduling algorithm in which:

- Priority CPU Scheduling executes processes based on priority.
- The process with the highest priority is executed first.
- Priority can be internal (time limit, memory, CPU usage) Or external (user importance, system requirements)
- Usually, smaller priority number = higher priority.
- When priorities of the process are equal (same), then the processes executed on FCFS (First-Come-First-Served) basis.

Types of Priority Scheduling

Non-Preemptive Priority Scheduling :-

Once the CPU is allocated, the process runs till completion.

Preemptive Priority Scheduling :-

CPU is taken away if a higher priority process arrives.

Advantages

- Important tasks get CPU first

- Flexible scheduling

Disadvantages

- Starvation possible
- Priority assignment is difficult

Important Formulae

Turnaround Time (TAT) :- $TAT = \text{Completion Time} - \text{Arrival Time}$

Waiting Time (WT) :- $WT = \text{Turnaround Time} - \text{Burst Time}$

PART A: Priority Scheduling with Different Priorities :-

Once the CPU is allocated, the process runs till completion.

Priority scheduling executes processes based on importance, not burst time

If a higher priority process arrives, the CPU is immediately allocated to it..

Given Data (Non-Preemptive) :-

Process	Arrival Time	Burst Time	Priority
P1	0	10	3
P2	0	1	1
P3	0	2	4
P4	0	1	2

(Lower number = Higher priority)

Execution Order :

P2 → P4 → P1 → P3

Gantt Chart :

P2	P4	P1	P3	
0	1	2	12	14

Calculation Table :

Process	BT	Priority	CT	TAT	WT
P2	1	1	1	1	0
P4	1	2	2	2	1
P1	10	3	12	12	2
P3	2	4	14	14	12

Average Waiting Time : $(0+1+2+12)/4=3.75$

Average Turnaround Time : $(1+2+12+14)/4=7.25$

PART B: Priority Scheduling with Equal Priorities

Rule for Equal Priority :

Processes with the same priority are scheduled using FCFS.

Given Data

Process	Arrival	Burst	Priority
P1	0	4	2
P2	0	3	2
P3	0	1	1
P4	0	2	3

Execution Order

P3 → P1 → P2 → P4

Gantt Chart :-

P3	P1	P2	P4	
0	1	5	8	10

Calculation Table :-

Process	BT	Priority	CT	TAT	WT
---------	----	----------	----	-----	----

P3	1	1	1	1	0
P1	4	2	5	5	1
P2	3	2	8	8	5
P4	2	3	10	10	8

Average Waiting Time : $(0+1+5+8)/4=3.5$

Average Turnaround Time : $(1+5+8+10)/4=6$

NSA